# VPC CHALLENGE

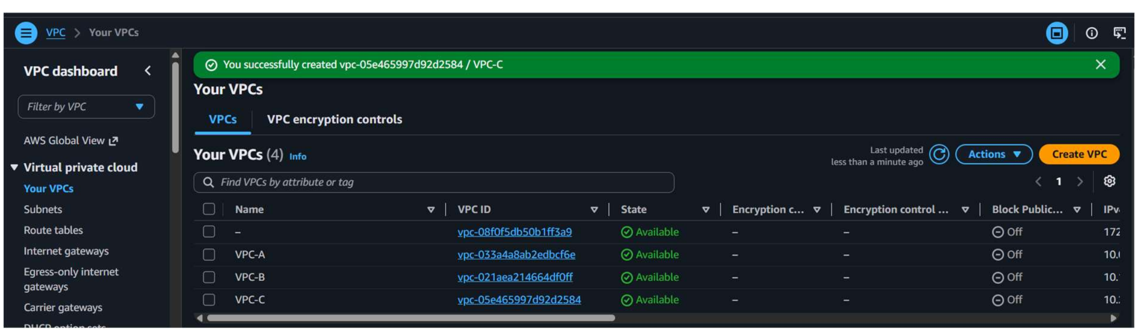
**Use Case: Setting up Transit Gateway and VPC Endpoints for a Multi-VPC Architecture**
**Scenario:**
**A large organization is migrating its on-premises infrastructure to the AWS cloud.**
**The organization's architecture involves multiple VPCs for different departments and applications, each requiring secure communication with centralized services and external resources.**
**The IT team needs to design and implement a scalable and efficient network architecture to accommodate the organization's growth and ensure robust connectivity between VPCs and external services.**
**Objectives:**

- **Design and deploy a scalable network architecture using AWS Transit Gateway to simplify network connectivity between multiple VPCs.**
- **Configure VPC endpoints to securely access AWS services without internet gateways or NAT gateways, ensuring data privacy and minimizing exposure to external threats.**

### Step 1: Create VPCs

Create three VPCs.

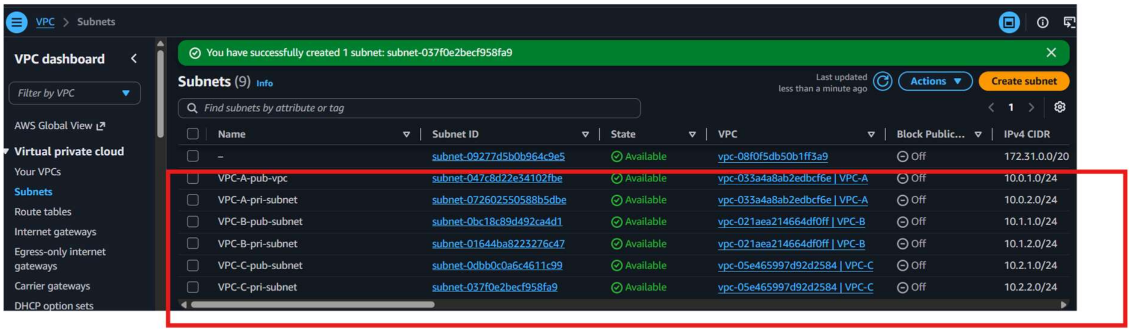
- VPC-A (10.0.0.0/16)
- VPC-B (10.1.0.0/16)
- VPC-C (10.2.0.0/16)




Create:

- 1 Public Subnet
- 1 Private Subnet

- Updated route table and Internet gateway



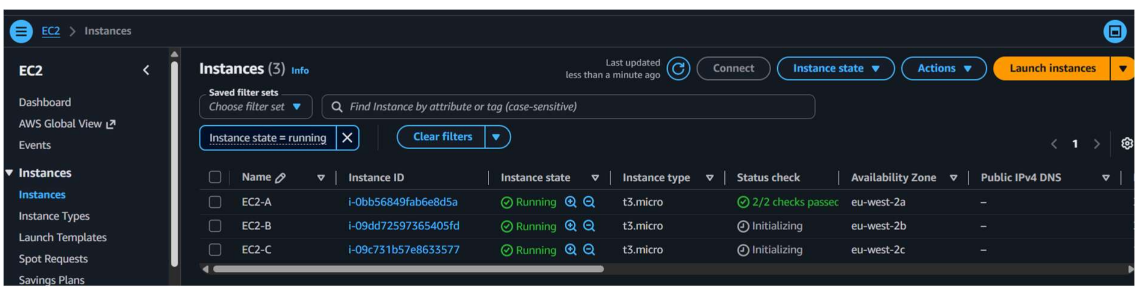

---

**Step 2: Launch EC2 Instances**

Launch one EC2 instance in each VPC.

Example:

- EC2-A
- EC2-B
- EC2-C

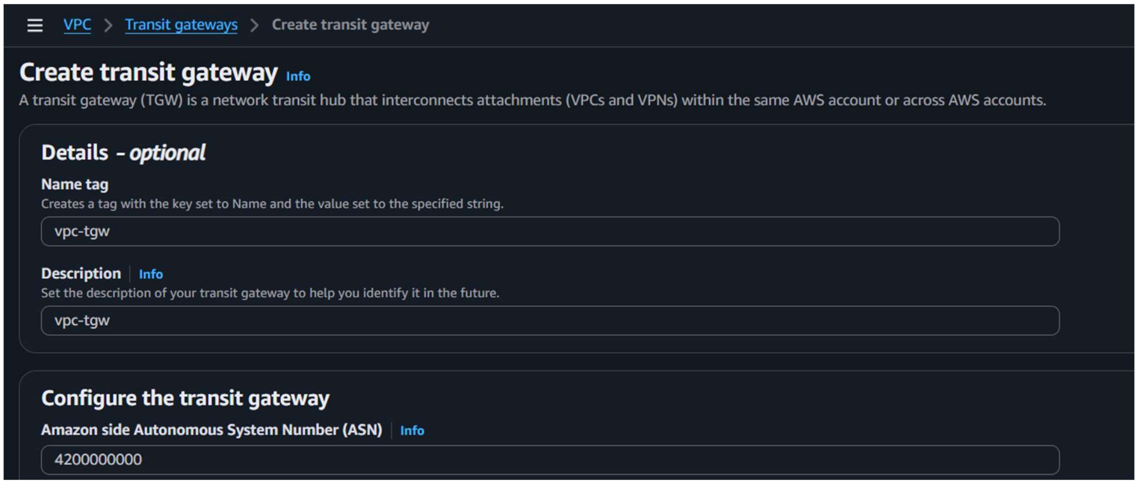



---

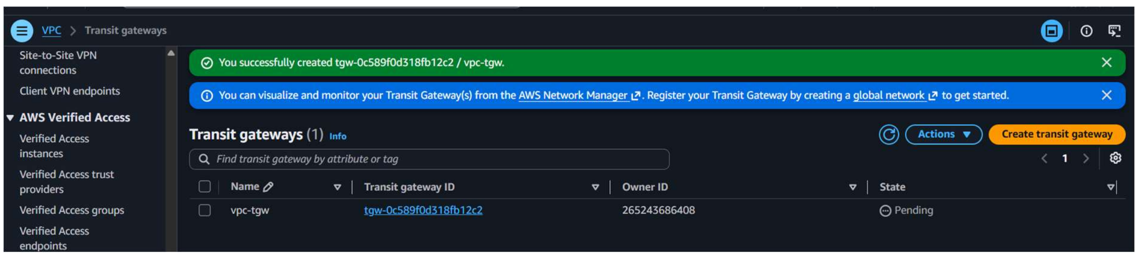
**Step 3: Create Transit Gateway**

Go to
**VPC → Transit Gateways → Create Transit Gateway**
Provide

- Name
- Amazon ASN (Default)
- Enable DNS Support




Click **Create**.




---

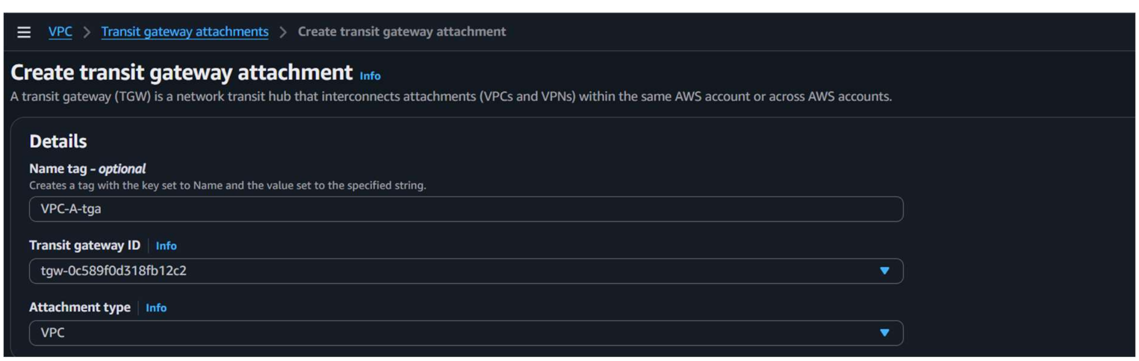
## Step 4: Attach VPCs to Transit Gateway

Go to

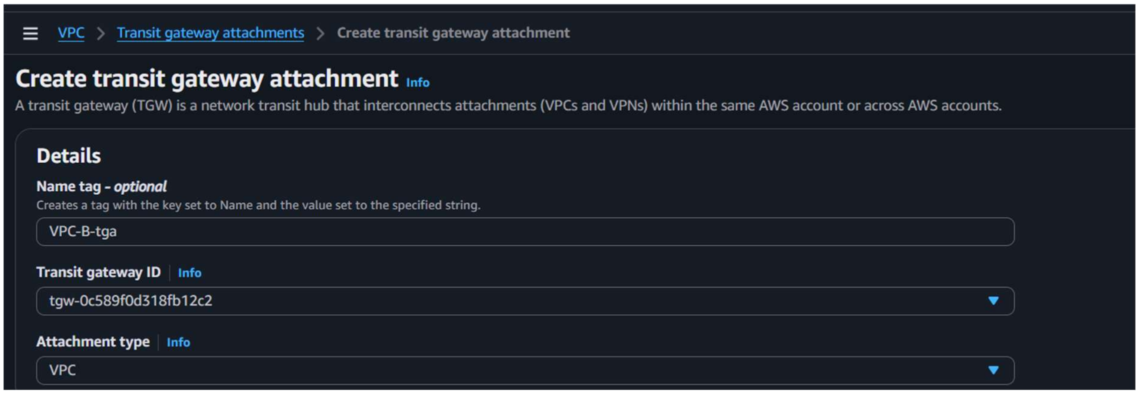
**Transit Gateway → Attachments → Create Attachment**

Attach:

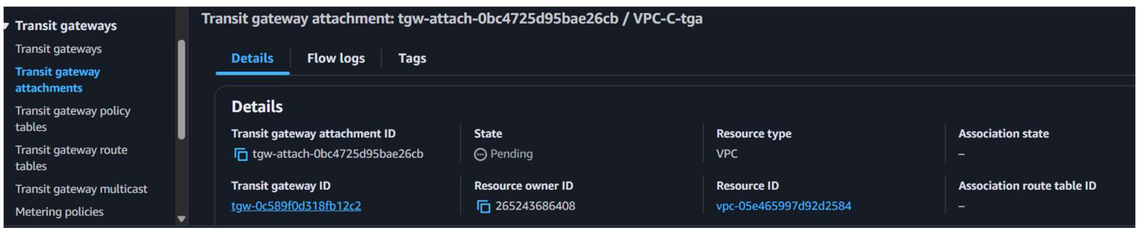
- VPC-A



- VPC-B

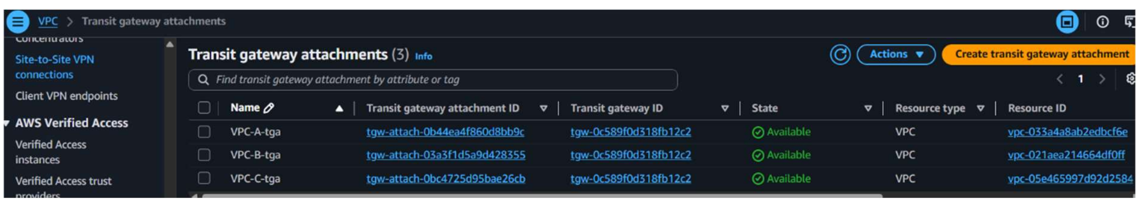



- VPC-C




Select the subnet in each VPC.

Wait until the attachment status becomes **Available**.




---

**Step 5: Update Route Tables**

In every VPC Route Table, add routes:

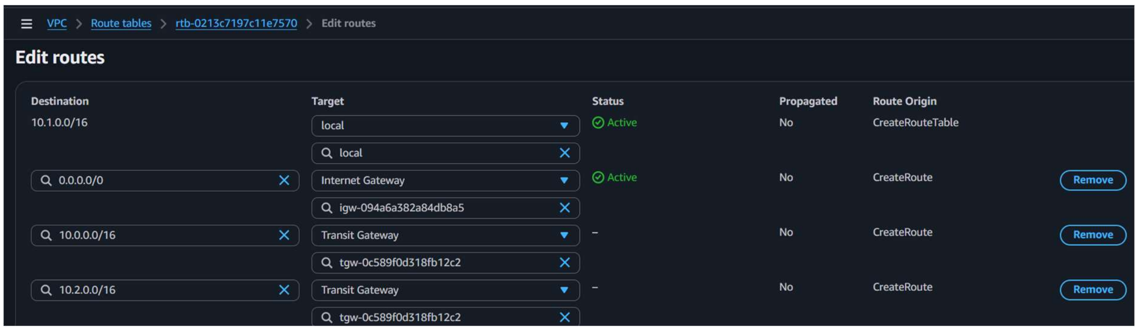
For VPC-A

| Destination | Target |
|---|---|
| 10.1.0.0/16 | Transit Gateway |
| 10.2.0.0/16 | Transit Gateway |


For VPC-B

10.0.0.0/16 Transit Gateway

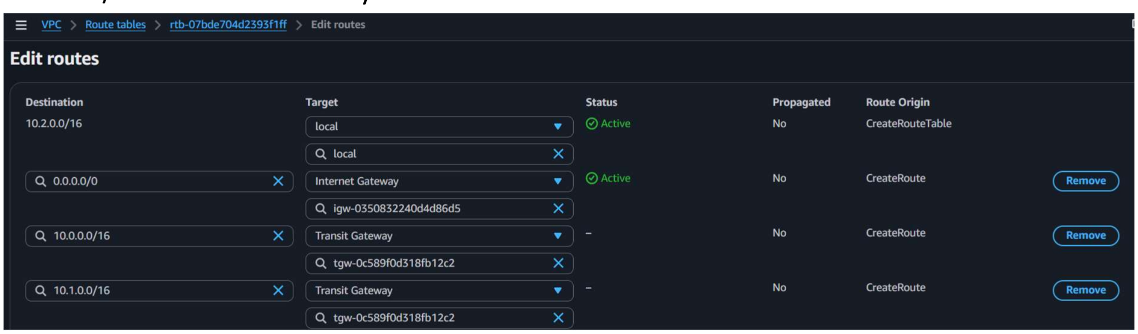
10.2.0.0/16 Transit Gateway




For VPC-C

10.0.0.0/16 Transit Gateway

10.1.0.0/16 Transit Gateway




---

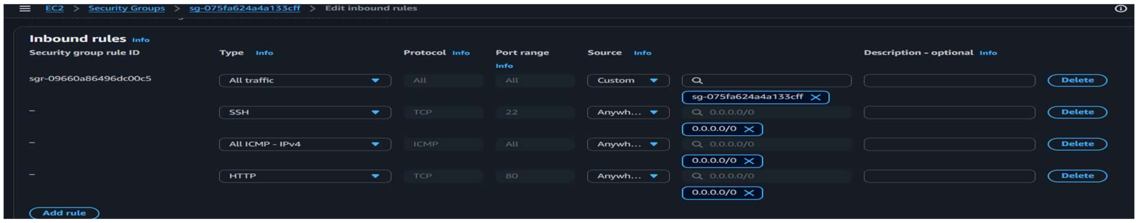
**Step 6: Configure Security Groups**

Allow ICMP (Ping)

or

Allow required ports like:

- SSH (22)
- HTTP (80)
- HTTPS (443)




---

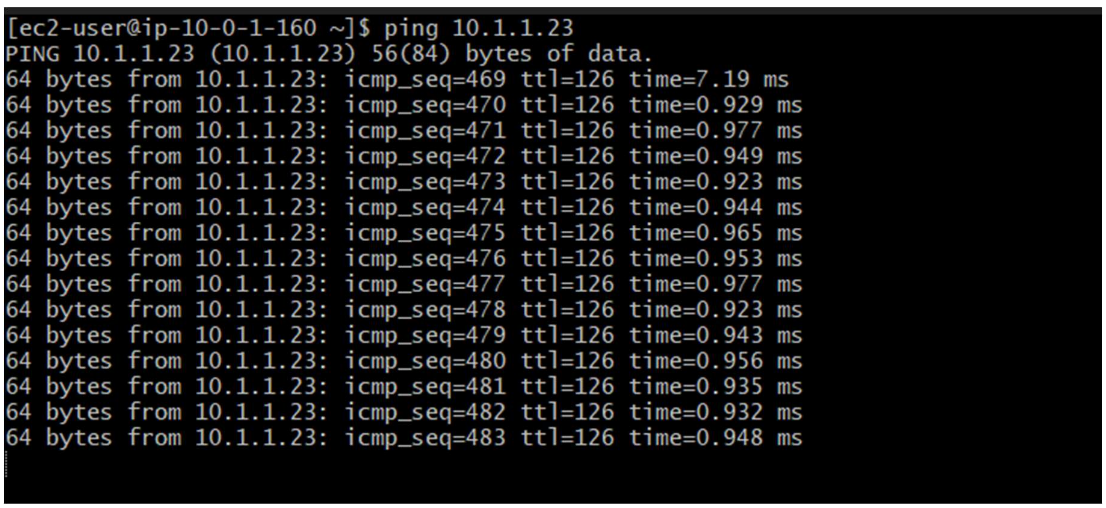
**Step 7: Test Connectivity**

From EC2-A

ping <Private-IP-of-EC2-B>

ping <Private-IP-of-EC2-C>

Connectivity should succeed through the Transit Gateway.



```
[ec2-user@ip-10-0-1-160 ~]$ ping 10.1.1.23
PING 10.1.1.23 (10.1.1.23) 56(84) bytes of data.
64 bytes from 10.1.1.23: icmp_seq=469 ttl=126 time=7.19 ms
64 bytes from 10.1.1.23: icmp_seq=470 ttl=126 time=0.929 ms
64 bytes from 10.1.1.23: icmp_seq=471 ttl=126 time=0.977 ms
64 bytes from 10.1.1.23: icmp_seq=472 ttl=126 time=0.949 ms
64 bytes from 10.1.1.23: icmp_seq=473 ttl=126 time=0.923 ms
64 bytes from 10.1.1.23: icmp_seq=474 ttl=126 time=0.944 ms
64 bytes from 10.1.1.23: icmp_seq=475 ttl=126 time=0.965 ms
64 bytes from 10.1.1.23: icmp_seq=476 ttl=126 time=0.953 ms
64 bytes from 10.1.1.23: icmp_seq=477 ttl=126 time=0.977 ms
64 bytes from 10.1.1.23: icmp_seq=478 ttl=126 time=0.923 ms
64 bytes from 10.1.1.23: icmp_seq=479 ttl=126 time=0.943 ms
64 bytes from 10.1.1.23: icmp_seq=480 ttl=126 time=0.956 ms
64 bytes from 10.1.1.23: icmp_seq=481 ttl=126 time=0.935 ms
64 bytes from 10.1.1.23: icmp_seq=482 ttl=126 time=0.932 ms
64 bytes from 10.1.1.23: icmp_seq=483 ttl=126 time=0.948 ms
```

---

**Step 8: Create VPC Endpoint**

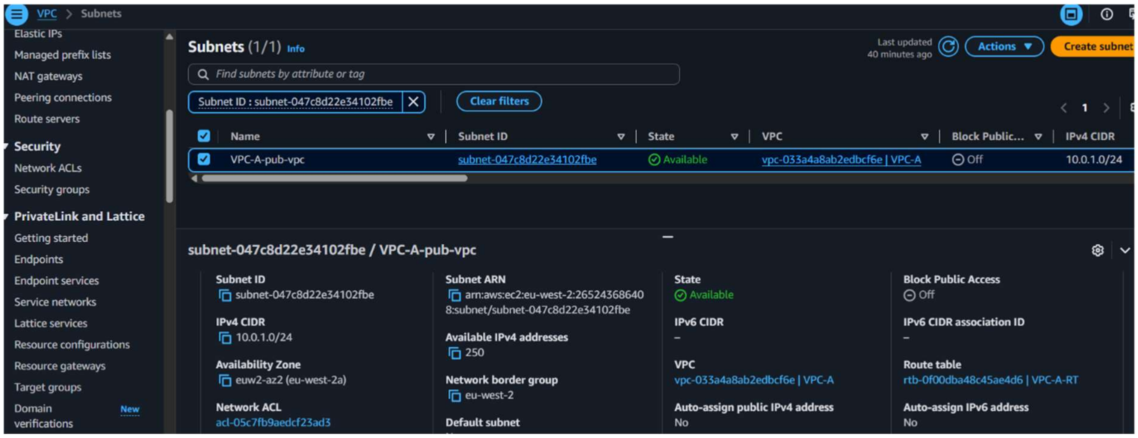
**VPC → Endpoints → Create Endpoint**

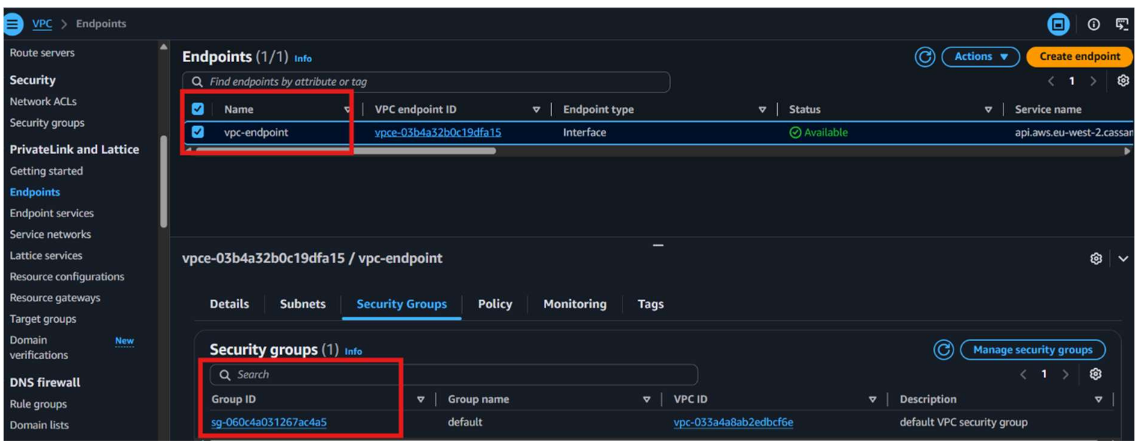
Choose the AWS service.

Example:

- S3 (Gateway Endpoint)
- DynamoDB (Gateway Endpoint)

- SSM (Interface Endpoint)




**Configure VPC endpoints to securely access AWS services without internet gateways or NAT gateways.**

An S3 Gateway Endpoint is specifically designed so EC2 instances can access Amazon S3 **without using an Internet Gateway or NAT Gateway**.